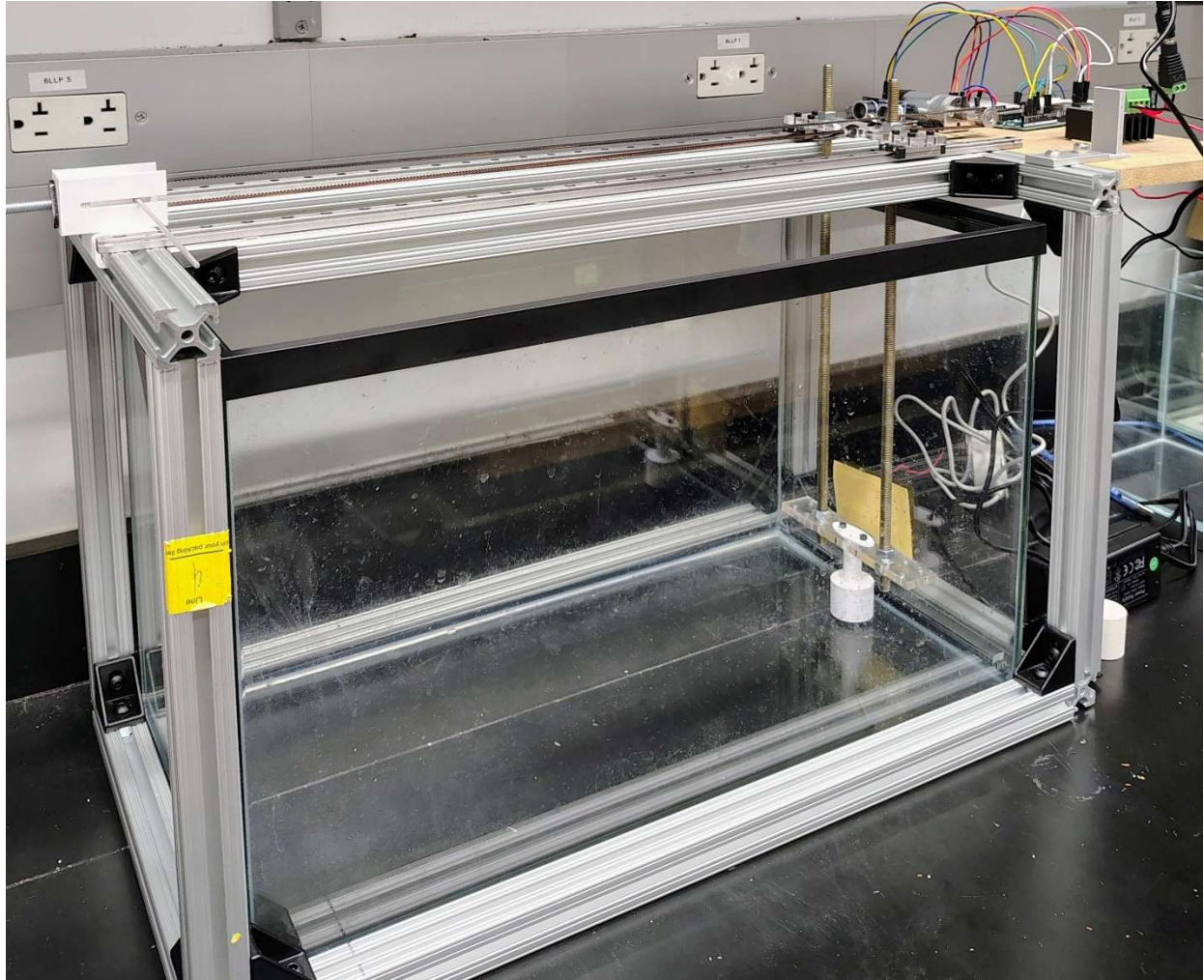


Algae Tow Tank Documentation & Notes

Jaine Ratush

2026



1. Purpose & Requirements

The TowTank is required to move a gantry at a constant speed to agitate bioluminescent algae through shear force, and create Karmen Vortices.

The required cruise speed was $v = 0.5ms^{-1}$ for at least 254mm.

2. Operation & Reset

- a. Plug the black cable(12V ACDC) into the wall
- b. Plug the white 9V ACDC cable into the wall
- c. The tank should do a pass
- d. To reset it:
 - Unplug the 9V white cable
 - Untension the belt and take it off from the driver pulley
 - Bring the gantry back to the starting position
 - Attach belt to driver pulley
 - Tension the belt(not too tight though)
 - Plug the 9V white cable in

Changing Velocity:

1. Open “Calibration_Kff_PWM”
2. Tools → Serial Plotter. Make sure the baud rate is 9600.
3. Unplug both cables
4. Change `int pwmVal=120; //PWM Calibration` to a higher value if increasing speed, and lower if decreasing.
Note: Max PWM value is 255 which corresponds to $v = 1.4 \frac{m}{s}$. It is not recommended to run the motor continuously over 230 PWM, unless current/torque limiting motion profiles or hardware is introduced to the tank.
5. Upload code to arduino
6. Plug motor 12V Black cable back into wall
7. Let the gantry do one pass & open the serial monitor. It will display the gantry velocities for this pass. Scroll to the middle of the new data to get a more accurate velocity.
8. Repeat 1-6 until you hit your target velocity.

Remember to power cycle the Arduino at the end of each pass/trial(unplug it at end of tank, and plug the Arduino back into your laptop once the gantry is at the starting position and you are ready for it move.)

9. Then switch to “*TowTank_Final*”
10. Unplug the 12V Black cable from the wall
11. Change `const float V_CRUISE` to your desired speed.
12. Change `int Kff =` to:
PWM Value that gets you the desired speed / Desired Speed
13. Upload the code to the Arduino
14. Unplug the Arduino from your laptop
15. Follow instructions a & b.

3. Components & Mechanical Design

a. Components:

- Structural
 - 1010 extrusions x8
 - Extrusion connectors & bolts x16
 - Fiber Platform
 - Motor Bracket
 - Mechanical
 - MGN9 Linear rail x2
 - Washers for Linear Rail fitting x14
 - 12V 4.4:1 Geared 25D Polulu DC Motor w/ 48CPR Encoder **
 - GT2 6mm wide timing belt
 - 3mm inner bore Idler pulley
 - 5mm inner bore, 12.2mm diameter driven pulley
 - Shaft adapter from 4mm to 5mm outerbore
- Pulley Tensioner:
- {1010 connection, Outer Slider, Inner Slider, M6 Hex Screw, 1010 rail slide-in fastener, 3mm rod, 3mm inner bore pulley}
- Gantry:
- {M8 Rods, Belt Connection Laser Print, Cylinder Holder Laser Print, M8 nuts x8, Zipties, Cylinder holder, Cylinder}

- Electrical

IBT2 Single H-Bridge Motor driver

Arduino Uno

HCS04 Ultrasonic sensor

12V AC/DC Adapter

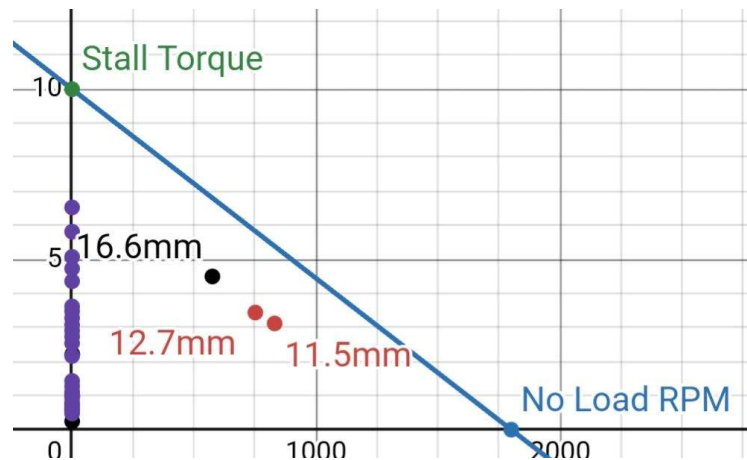
9V ACDC Adapter

b. Mechanical Design

- 12V Polulu 4.4:1 Planetary Gear Motor w/ Encoder &
12.2 mm Driven Pulley GT2 Curvilinear
2200 cm Open GT2 Curvilinear Belt
Pulley to Motor Adapter

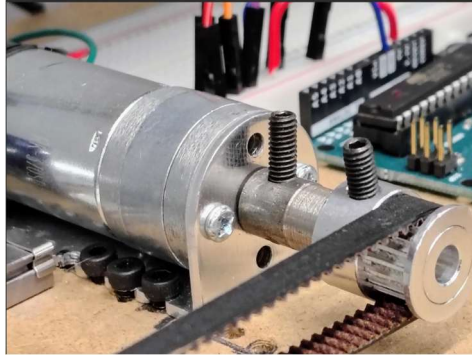
The Motor needed to be able to reach $v = 0.5ms^{-1}$ at close to half of the Speed-Torque curve of a DC motor.

<https://www.desmos.com/calculator/bqttl8z7cw>



Using desmos, the Torque-Speed “curve” of a DC motor was plotted. On the same graph points where (x=RPM required, y=Torque required) for each driven pulley size were plotted. At the midpoint of the line, the motor is most efficient. The system would have most flexibility and overhead room, so 12.2mm pulley diameter was chosen.

For precision a curvilinear GT2 precision pulley was chosen. However, they were only available in a 5mm bore or 3mm bore, and the motor has 4mm D-Shaft, so an adapter was machined in the shop using a manual Lathe & Mill.



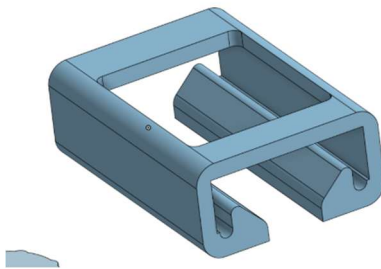
Motor Assembly



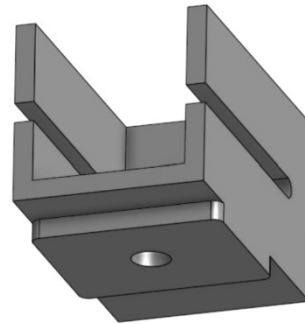
Adapter

- Belt Tensioner {M6 Hex screw, T-rail Sliding nut}
Idler Pulley

The belt tensioner and Idler pulley were compressed into one assembly. To tension the belt, the idle pulley would be pulled back by converting rotational motion to linear motion.



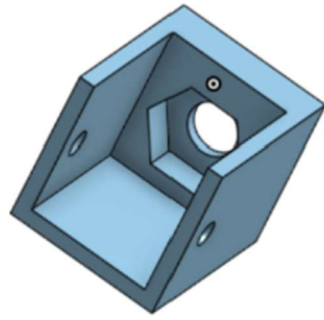
Tensioner Base



Slider Case

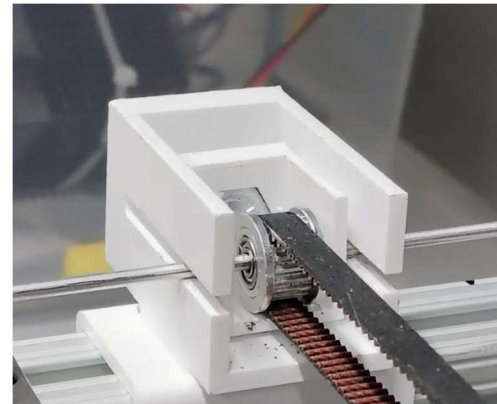
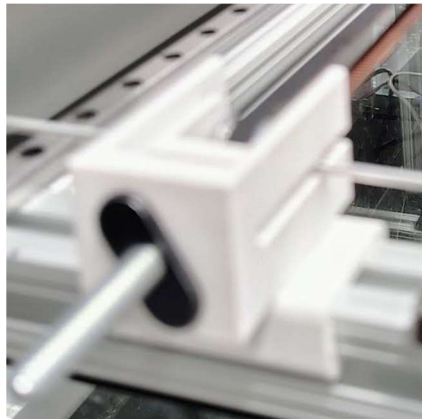
The tensioner base was designed to slide onto the 1010 extrusion to avoid shearing bolts and has thick walls/ geometry to withstand forces from the belt. A gyroid infill was used so that shear forces would not split the PLA layers.

The slider case is a tight press fit into the base to withstand forces from the belt and has a screw to restrict movement along the 1010 rail.



Slider

Assembled:



Rotational motion was converted to linear motion by fixing a screw in place, and turning a nut at the back end of the slider case.

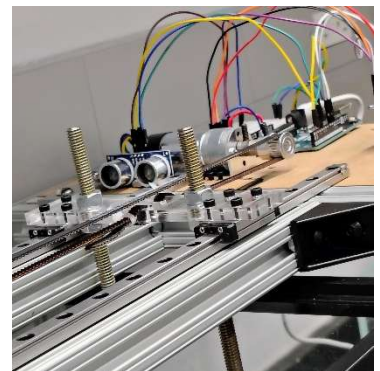
The hex screw was fixed in place using a tight-fit indent for the screw head. Credit to Zara Bhuiyan for hex indent.

- Gantry{Belt Slotter, Cylinder Rod holder, MGN9 Carriages, M8 Threaded Rods & Nuts}



The tank is about 2 feet high, however, the fluid level is only 4" high. Deflection of the cylinder was the main concern. Using a similar method as to the belt tensioner, the cylinder holder was made height adjustable. The nuts keep the subsystem rigid, and short height of the cylinder reduces noticeable deflection in the fluid/algae/organism.

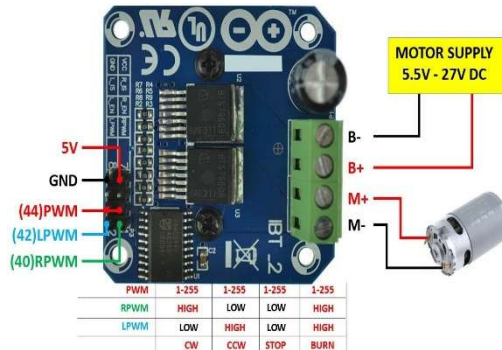
The cylinder is detachable from the cylinder rod (skinny white part that goes through acrylic bottom piece) so that cylinder diameter and shape can be changed for different use cases.



4. Electrical Design

a. Component explanation

- BTS7960(IBT2) Single H-Bridge Motor Driver-



The motor's stall current is 5A.
The driver has a continuous current range of up to 30A.

Uses MOSFETS to adjust motor speed and direction through PWM.

PWM(Pulse Width Modulation): A full voltage source is pulsed on and off with frequency/Duty Cycle commanding desired average voltage to the motor. Reduces heating that comes with trying to decrease to some average or specific voltage.

- 48CPR Quadrature Encoder:



Lead Color	Function
Red	Motor power M1
Black	Motor power M2
Green	Encoder ground
Blue	Encoder power (3.5 V to 20 V)
Yellow	Encoder channel A output
White	Encoder channel B output

Used for position and velocity data.
Contains a disc that has alternating magnetic poles. As it rotates a hall effect sensor senses the changes in the magnetic field. Every time North is registered, that corresponds to square wave A. If South is registered, that corresponds to square wave B.

The rising method, used in the code works like this:

If B rises before A, the motor is rotating ccw, and encoder counts increase positively. The same thing happens if A and B are reversed, except signs & motor direction change.

However, since this a quadrature encoder, counting rises and falls of A and B is possible, and you can increase accuracy up to 211.2 CPR instead of 48CPR through code.

The code must be able to interrupt the loop to get encoder data, so channels A & B must be connected to pins 2 & 3 on the Arduino.

- HCS04 Proximity Sensor

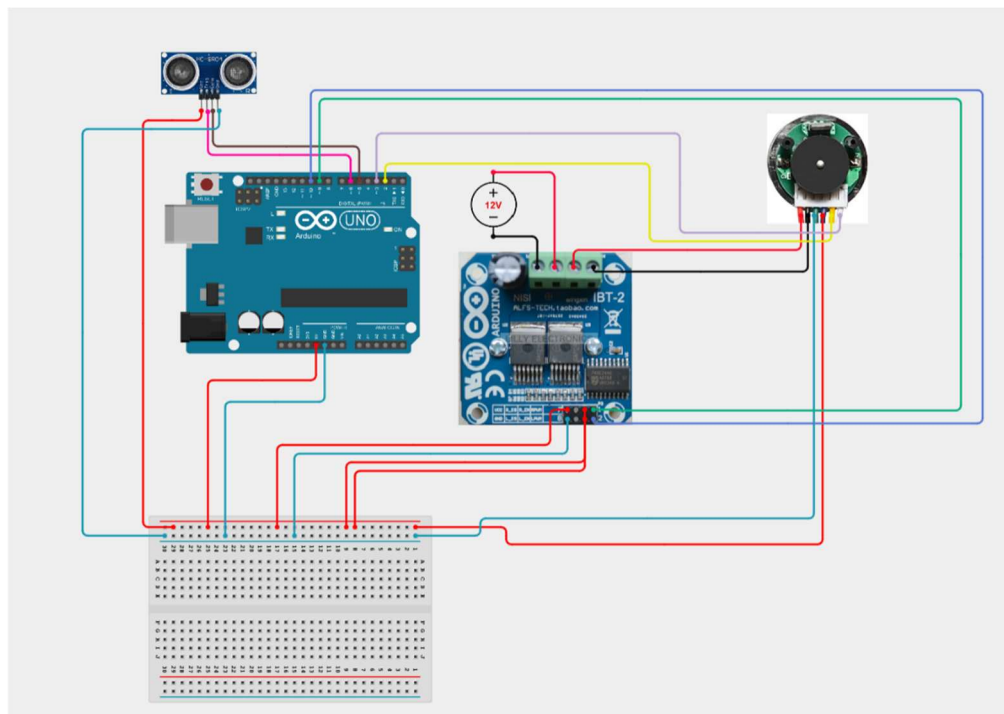


Used instead of a limit switch due to being more compact and having no wires go from one end of the tank to the other.

The trigger pin generates a wave which leaves the cone of the sensor. Once the wave bounces off the object, the echo pin registers. The time between echo & trigger activation allows for object distance to be calculated.

Trig & Echo are digital, meaning they're either HIGH or LOW(Registered/Not registered), and must be connected to the digital pins on the Arduino.

b. Overall Schematic:



5. PI Controller, Feedforward & Programming

The tank uses a closed feedback loop with PI control and a feedforward signal to achieve an accurate desired velocity.

PI controller: Uses a correction factor proportional to error (P) K_p and an integral factor (I) K_i that adds up error over time to correct small differences between target and actual velocity. The velocity on the tank has average error of 0.03m/s.

The feedforward term K_{ff} prevents the integral correction feedback from accumulating too much as the motor ramps up. Then the PI controller makes small corrections to achieve your accurate desired velocity.

Without the feedforward, the motor takes a long time to startup, as the PI controller is the only thing governing the commanded PWM.

“Calibration_Kff_PWM” Code:

Calibrating PWM allows you to correlate an velocity approximate to your velocity. This this code takes PWM as an input, and outputs the correlated velocity.

```
int pwmVal=120; //PWM Calibration corresponds to v=0.5m/s.
```

“Tow_Tank_Final” Code:

$$K_{ff} = \frac{PWM\ Value}{Desired\ Velocity}$$

Once K_{ff} is empirically determined, and K_{ff} is entered into the code, remember to change `const float V_CRUISE` to the desired velocity.

As of now the tank is set to do one forward pass across the tank, and must be manually reset according to instructions above. However,

trapezoidal and forward-back motion can be enabled, by uncommenting the trapezoidal motion section.

6. Improvements & Limitations

Limitations:

The motor is limited to a maximum 2200RPM, and has a stall current of 5A.

The gantry's nuts need to be tightened occasionally, as vibration causes them to loosen which introduces deflection to the cylinder.

Future Improvements:

a. Mechanical

- Gantry is height adjustable, but was not designed for constant and easy adjustment. Mini Tow Tank 2.0 will have one threaded rod, and one smooth rod with clamps, allowing for faster adjustment.
- The Laser printed top and bottom portion of the gantry can be re-laser cut such that the nuts sit farther away from the MGN9 carriages to reduce nut-loosening vibrations.
- In a future iteration the Go Pro should be able to be mounted on the gantry

b. Controls/Electrical

For the next iteration the following improvements could be made:

- Add a homing button and a stop button
- Adding capacitor between motor & breadboard to prevent back emf from resetting if the belt is not removed from the driver pulley
- Using an absolute encoder instead of the current relative encoder
- Better wire management and casing for electronics